



Polimorfismo: un mensaje, varias implementaciones

El problema

Al cierre del día, el local necesita listar todos los cobros y sumar cuánto se acredita en la cuenta. Los pagos están guardados en tres listas (ArrayList), una por medio de pago, y el cierre de caja se escribió así:

```
double total = 0;

for (PagoDebito pago : debitos) {
    pago.mostrarComprobante();
    total += pago.montoAcreditado();
}

for (PagoCredito pago : creditos) {
    pago.mostrarComprobante();
    total += pago.montoAcreditado();
}

for (PagoQR pago : qrs) {
    pago.mostrarComprobante();
    total += pago.montoAcreditado();
}
```

Tres bucles idénticos salvo por el tipo de la variable. El problema es le mismo de siempre, el día que el local acepte transferencias hay que agregar una lista, un cuarto bucle y acordarse de sumarlo al total.

Ahora tenemos que aplicar lo que aprendimos en las dos clases previas. Los tres son un Pago. Los tres tienen montoAcreditado() y mostrarComprobante(), porque Pago los declara. Lo único que falta es poder escribir un solo bucle que no pregunte de qué tipo es cada uno.

Una variable de un tipo, un objeto de otro

Esta línea compila:

```
Pago pago = new PagoQR(1, venta, "PAGO;lapampa.kiosco;ARS");
```

La variable se declara Pago y el objeto que se le asigna es un PagoQR. Se puede porque un pago con QR **es un** pago, la misma relación que justificaba la herencia. Al revés no funciona, no todo pago es un pago con QR.

Ahí conviven dos tipos distintos:

- El **tipo estático** es el de la variable, Pago. Es lo único que ve el compilador.
- El **tipo dinámico** es el de la cosa que existe en memoria, PagoQR. Es lo que hay en tiempo de ejecución.

La consecuencia de declarar como Pago es que el compilador solo deja llamar lo que declara esta en Pago:



```
pago.mostrarComprobante(); // compila: Pago lo declara
pago.armarSolicitudCobro(); // NO compila, aunque el objeto sea un PagoQR
```

armarSolicitudCobro() existe en el objeto porque es un PagoQR, pero el compilador no lo sabe: lo único que le consta es que la variable es un Pago, y Pago no tiene ese método. La referencia solo ve la parte del objeto que su propio tipo declara.

El enlace dinámico

La pregunta interesante es la otra. Si la variable es un Pago, cuando se escribe esto:

```
pago.mostrarComprobante();
```

¿qué versión se ejecuta, la de Pago o la que PagoQR sobrescribió?

Se ejecuta la de PagoQR. El compilador solo verificó que el método existiera, cuál de las versiones corre se decide recién en ejecución, mirando el tipo dinámico. Ese mecanismo se llama **enlace dinámico o binding dinámico**, y es lo que hace que la sobrescritura de la clase 5 sirva para algo más que reutilizar código.

El compilador mira la variable. La ejecución mira el objeto.

Eso es el polimorfismo, el mismo mensaje “mostrarComprobante()” provoca respuestas distintas según quién lo reciba, y quien lo envía no necesita saber quién es.

El cierre de caja, otra vez

Con eso, los tres bucles se convierten en uno:

```
public static double cerrarCaja(ArrayList<Pago> pagos) {
    double total = 0;
    for (Pago pago : pagos) {
        pago.mostrarComprobante(); // cada uno imprime el suyo
        total += pago.montoAcreditado(); // cada uno retiene lo suyo
    }
    return total;
}
```

Vale la pena mirar lo que este método no tiene. No tiene un if preguntando de qué tipo es cada pago. No nombra a PagoDebito, ni a PagoCredito, ni a PagoQR. No sabe cuántos medios de pago existen. Y por eso, el día que se sume PagoTransferencia, no hay que tocarlo, alcanza con que la clase nueva herede de Pago y escriba su montoAcreditado().

Ese es el beneficio, el código que usa la abstracción no cambia cuando aparecen implementaciones nuevas.

Lo mismo sucede con interfaces. El método anularTodo(Anulable[] operaciones) de la clase pasada ya era polimórfico: recorría pagos con QR, débitos y ventas sin distinguirlos, y cada anular() hacía algo distinto. La única diferencia es que ahí el tipo común venía de un interfaz (o contrato) y no de una superclase.



El otro polimorfismo: la sobrecarga

Todo lo anterior es **polimorfismo dinámico**, que se resuelve en ejecución. Java tiene además el **polimorfismo estático**, más conocido como **sobrecarga de métodos**, varios métodos con el mismo nombre en la misma clase, que se distinguen por su lista de parámetros.

```
public void registrar(Pago pago) { ... }  
public void registrar(Pago pago, String observacion) { ... }
```

Ya lo vimos, por ejemplo `System.out.println` viene sobrecargado para aceptar un `String`, un `int` o un `double`, y por eso funciona con cualquier cosa que se le pase. Varios constructores de una misma clase son el mismo mecanismo.

La diferencia entre los dos está en quién decide y cuándo:

	Sobrecarga (estático)	Sobrescritura (dinámico)
Dónde	Varios métodos en la misma clase	El mismo método en la superclase y en la subclase
Qué cambia	La lista de parámetros	Nada: la firma es idéntica
Quién decide	El compilador, mirando los argumentos	La máquina virtual, mirando el objeto
Cuándo	Al compilar	Al ejecutar

Los dos se llaman polimorfismo, pero el más importante en el diseño orientado a objetos es el dinámico.

Cuando hace falta lo específico

A veces el código sí necesita algo que solo existe en una subclase. Para eso están `instanceof`, que pregunta por el tipo dinámico, y el casting, que le promete al compilador que la referencia se puede tratar como algo más específico:

```
if (pago instanceof PagoQR) {  
    PagoQR qr = (PagoQR) pago;  
    qr.armarSolicitudCobro();  
}
```

Desde Java 16 se puede escribir más corto, declarando la variable en la misma pregunta:

```
if (pago instanceof PagoQR qr) {  
    qr.armarSolicitudCobro();  
}
```

Ahora, la advertencia. Cada `instanceof` adentro de un bucle es una pregunta que el objeto podría estar respondiendo solo. Si aparecen tres seguidos, uno por cada subclase, eso no es



polimorfismo, es el mismo `if` encadenado de siempre, escrito con otra sintaxis, y el método que falta va en `Pago`. El usar el `instanceof` da la pista de que el polimorfismo no existe o la herencia o la interfaz esta mal definida.

Dos cosas que no son polimórficas

El enlace dinámico vale solo para los métodos de instancia.

Los **atributos** no se sobrescriben. Si la superclase y la subclase declaran un atributo con el mismo nombre, conviven los dos, y el que se usa depende del tipo de la variable, no del objeto. Es una fuente de errores silenciosos, y es otra razón para lo que se viene sosteniendo desde la clase 3: los atributos van `private`, y se llega a ellos por métodos.

Los métodos **static** tampoco. Pertenecen a la clase, no al objeto, así que también se resuelven por el tipo de la variable. Un método `static` no se sobrescribe, si la subclase declara uno con la misma firma, lo que hace es taparlo, no reemplazarlo.

Bibliografía sugerida para profundizar

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*.

- Capítulo 10: Object-Oriented Programming: Polymorphism and Interfaces. Arranca con el mismo argumento de esta clase, el código que no cambia cuando aparecen subclases nuevas, y trata el enlace dinámico y el casting con `instanceof`.

Aguilar, L. J. (2011). *Programación en Java*. McGraw-Hill.

- Capítulo 14 completo, dedicado al polimorfismo: la sección 14.1 lo define y lo relaciona con la herencia, y la 14.2 (p. 344) lo desarrolla con clases abstractas.
- Capítulo 8, para la sobrecarga de métodos y de constructores dentro de una misma clase.

García Llinás, L. F. (2010). *Todo lo básico que debería saber sobre Programación orientada a objetos en Java*. Ediciones Uninorte.

- Capítulo 1, sección 1.11 (p. 96) y las páginas 105 y 106, donde el polimorfismo aparece junto a las clases abstractas y el enlace dinámico.